Copyright All rights reserved

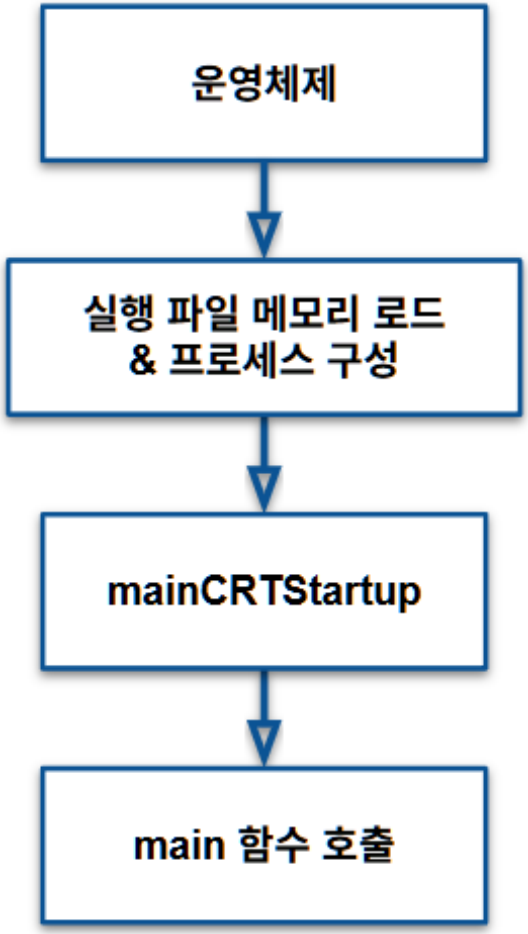
# 02. 프로그램 구성과 main 함수

Heesung Oh
https://github.com/3dapi

# 2.1 main 함수 호출 과정

Heesung Oh
Copyright All rights reserved

- Windows OS에서 main 함수가 호출되는 과정
  - 운영체제 → CRT 초기화 → main 호출 → CRT 종료 → 운영체제



# 2.2 main 함수의 다양한 형태

Heesung Oh
Copyright All rights reserved

- C 표준 main 함수

```
int main(void)
int main(int argc, char** argv) 또는
int main(int argc, char* argv[])
```

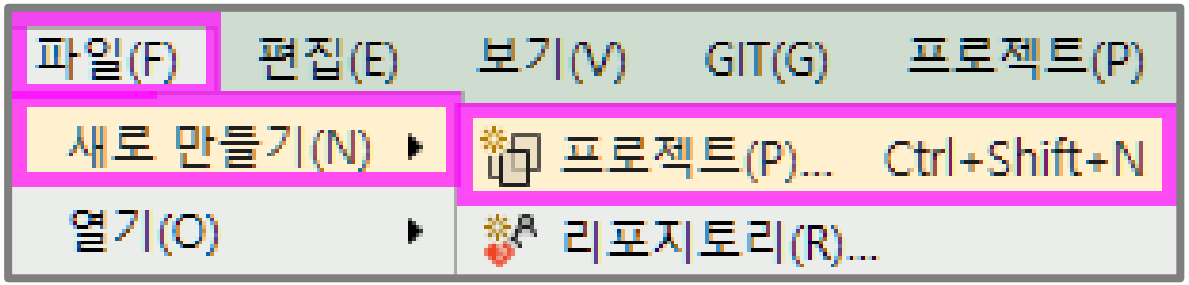
- 비 표준 main 함수

```
void main(void)
void main(int argc, char" argv) 또는
void main(int argc, char* argv[])
```

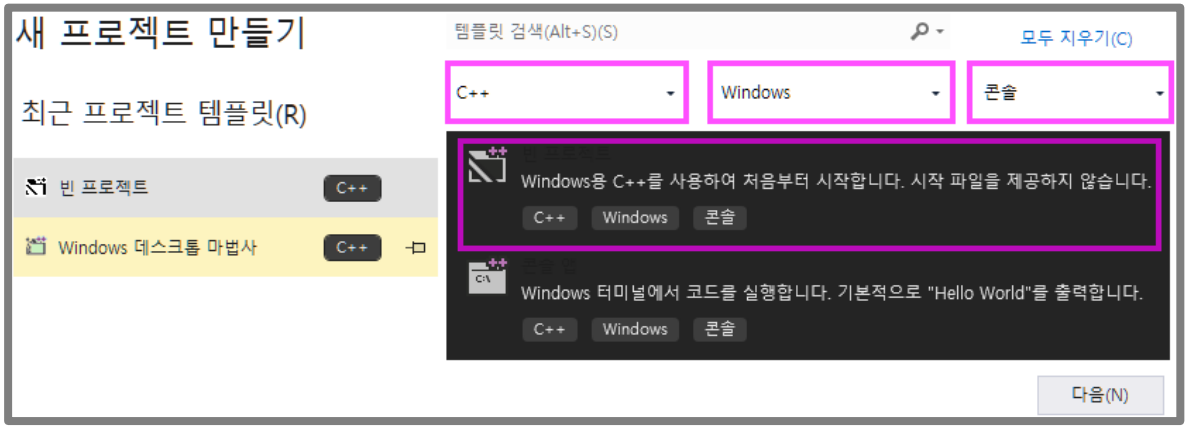
# 2.3 프로젝트 생성: myLesson

Heesung Oh
Copyright All rights reserved

- **프로젝트 생성**
  - [메뉴] → [파일] → [새로 만들기] → [프로젝트]

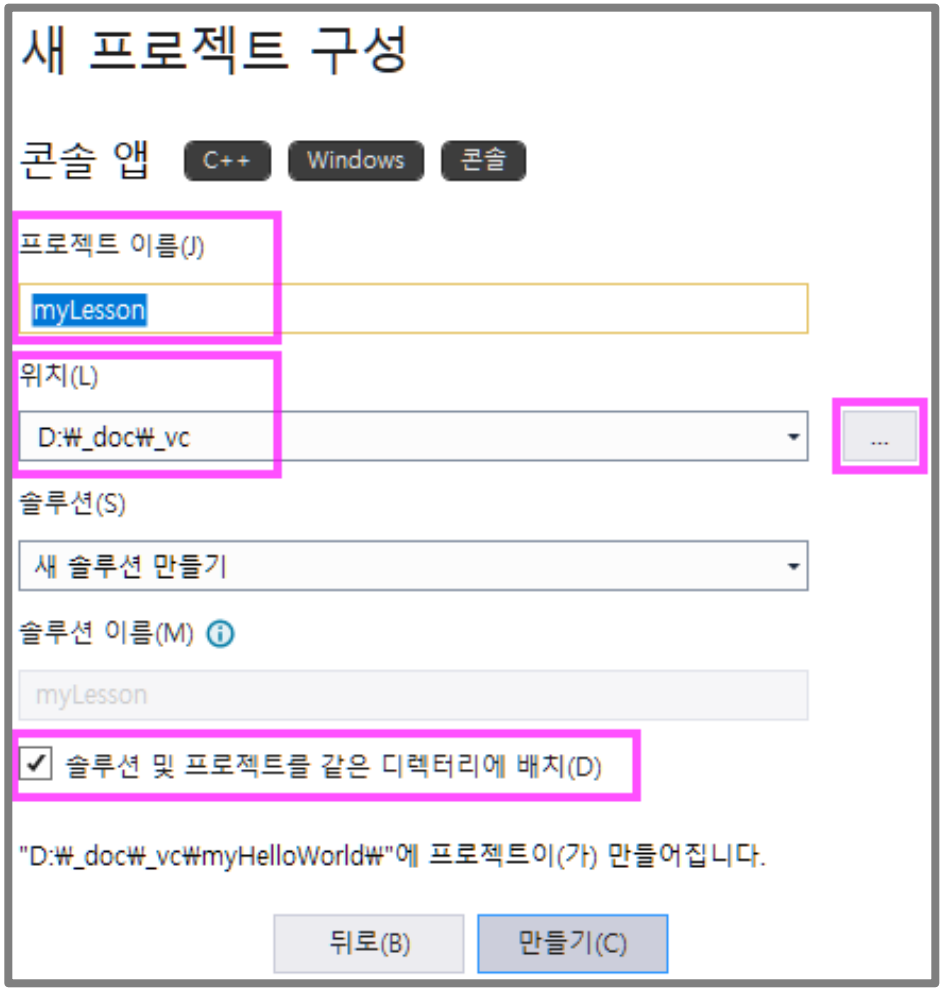



- **새 프로젝트 생성**
  - 시작 파일 없는 프로젝트 선택



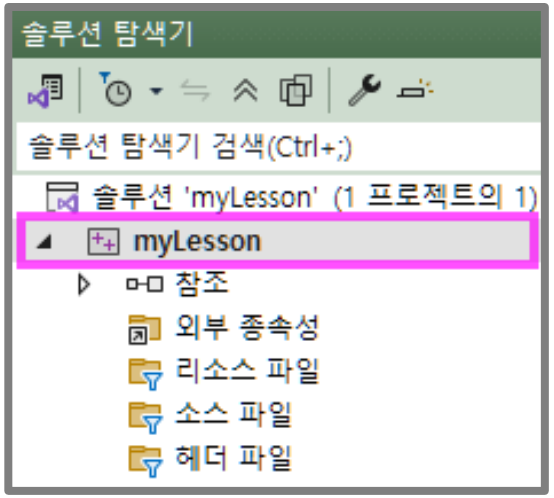
Heesung Oh
Copyright All rights reserved

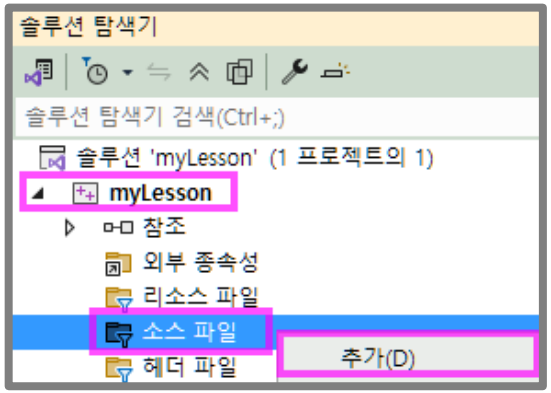
- 새 프로젝트 구성: myLesson

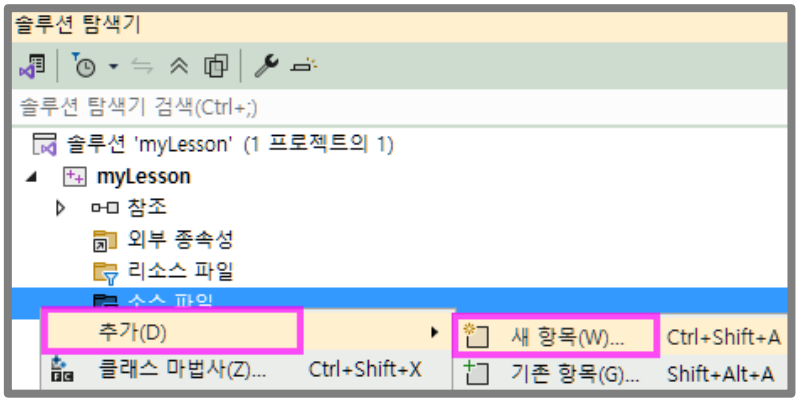


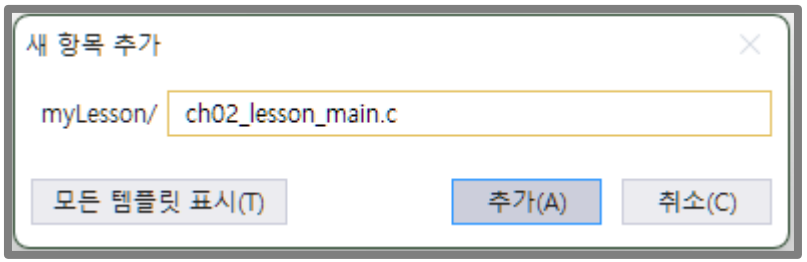
Heesung Oh
Copyright All rights reserved

- **소스 파일 추가**



# 2.3 프로젝트 생성: myLesson

Heesung Oh
Copyright All rights reserved

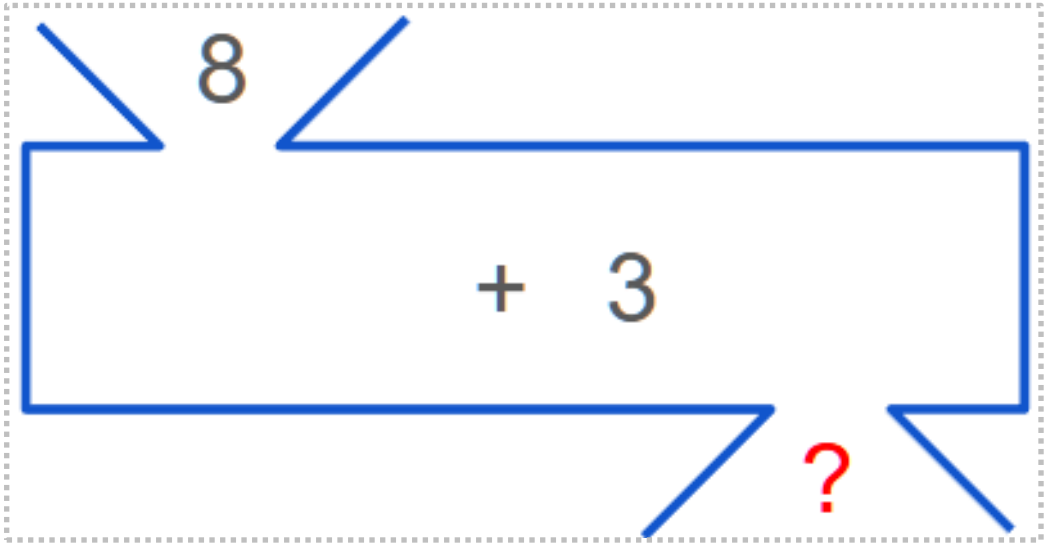
- 가장 간단한 C 프로그래밍

```
int main(void)
{
    return 0;
}
```

```
int main(int argc, char** argv)
{
    return 0;
}
```

Heesung Oh
Copyright All rights reserved

- 함수란?
  - 기능을 수행하는 장치 ➔
    - 기능을 수행하는 프로그램 단위
  - 수학적 함수: 입력 → 계산 → 결과 ➔
    - 입력 → 실행 → 결과



Heesung Oh
Copyright All rights reserved

- **C 언어 함수 원형**

```
반환형 함수_이름(인수 목록)
{
    명령어;
    return 반환값;
}
```

- **반환 없을 때:**

```
void 함수_이름(인수 목록)
{
    명령어;
    return; // 보통 생략함
}
```

# 2.4 프로그래밍 구성과 함수

Heesung Oh
Copyright All rights reserved

- 반환형
  - 함수 실행 후 반환되는 값의 자료형
- 함수 이름
  - 함수를 식별하고 호출하기 위한 이름
- 인수 목록
  - 함수로 전달되는 값을 저장하는 변수 목록
- 명령문
  - 함수 내부에서 실행되는 프로그램 명령
- return
  - 함수 실행을 종료하고 값을 반환
  - 반환 값이 없으면 반환 값 생략. → return ;
- 반환값
  - 함수가 호출한 곳으로 전달하는 값
- 세미콜론
  - 명령문의 끝을 나타내는 기호

# 2.5 컴파일과 빌드

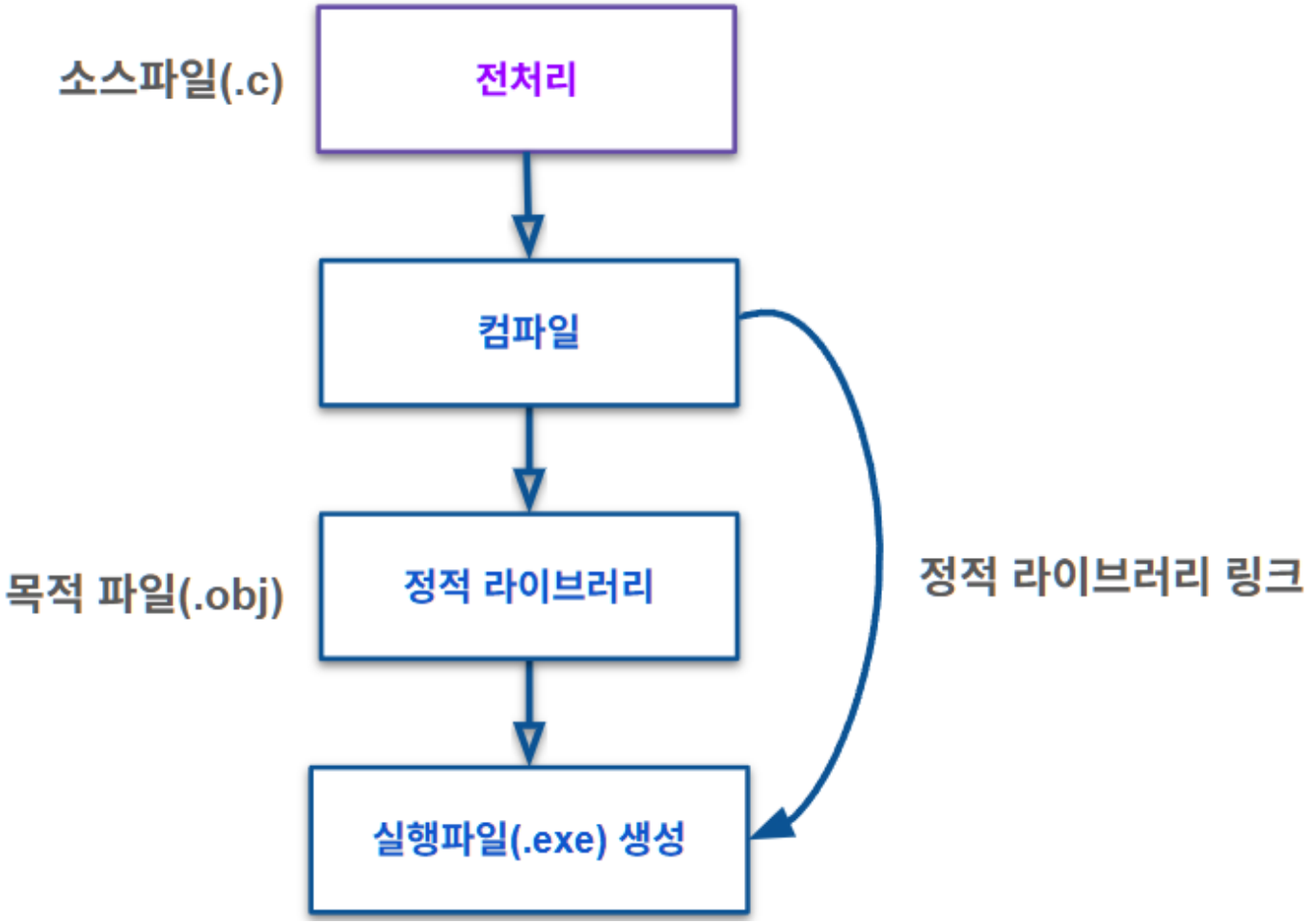
Heesung Oh
Copyright All rights reserved

- 넓은 의미의 컴파일
  - 전처리 → 컴파일 → 링크 → 실행 파일 생성
  - 전체 빌드 과정

- 좁은 의미의 컴파일
  - 전처리 → 컴파일 → 기계어 목적 파일(.obj)

Heesung Oh
Copyright All rights reserved

- **프로그램 빌드 과정**



# 2.5.1 전처리(Preprocessing) 단계

Heesung Oh
Copyright All rights reserved

- 전처리문
  - "#" 으로 시작. Directive
- #include 처리
  - 지정된 헤더 파일의 내용을 현재 소스 코드에 삽입
- 매크로 확장(#define)
  - 정의된 매크로를 실제 값이나 코드 형태로 치환
- 조건부 컴파일(#if, #ifdef, #ifndef, #elif)
  - 지정된 조건에 따라 특정 코드 블록을 포함 또는 제외
- 주석 제거
  - 주석 삭제
- 기타 지시문 처리(#pragma, #line 등)
  - 컴파일러에 전달할 추가 지시나 설정을 처리

Heesung Oh
Copyright All rights reserved

- Visual Studio에서 전처리 결과 확인
  - 프로젝트 속성 → C/C++ → 전처리기 → 파일로 전처리 (Preprocess to a File) → 예(/P)

| | | |
|---|---|---|
| ◢ C/C++ | 전처리기 정의 해제 | |
| 일반 | 모든 전처리기 정의 해제 | 아니요 |
| 최적화 | 표준 포함 경로 무시 | 아니요 |
| 전처리기 | 파일로 전처리 | 예(/P) |
| 코드 생성 | 전처리 줄 번호 표시 안 함 | 아니요 |

```
main.c
  ↓ 전처리
main.i
```

Heesung Oh
Copyright All rights reserved

- **전처리 전후 비교**

전처리문 정의

```
#define SQUARE(x) ((x)*(x))

int main()
{
    SQUARE(4);
```

전처리문 사용

```
int main()
{
    ((4)*(4));
```

치환된 전처리문

# 2.5.2 컴파일(Compilation) 과정

Heesung Oh
Copyright All rights reserved

- 중간 표현(Intermediate Representation, IR)
  - 최적화를 위해 사용하는 내부 코드 표현

- 최적화(Optimization)
  - 불필요한 연산 제거 및 코드 개선
  - 최적화 옵션

```
-O0(/Od) 디버깅용
-O1(/O1) 기본 최적화
-O2(/O2) 속도 최적화(릴리즈 빌드)
```

- 기계어 변환(Code Generation)
  - CPU 아키텍처(x86, ARM 등)에 맞는 기계어 생성

# 2.5.3 링킹(Linking)

Heesung Oh
Copyright All rights reserved

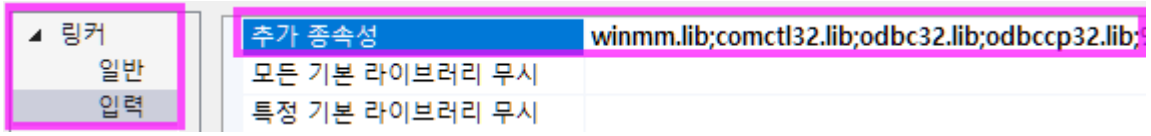
- 링커(Linker)
  - 목적 파일(.obj) + 정적 라이브러리 결합 → 실행 프로그램(.exe) 생성
- 정적 링크(Static Linking)
  - 프로그램 실행에 필요한 외부 코드(라이브러리)를 실행 파일 내부에 포함
- 심볼 해결(Symbol Resolution)
  - 함수와 변수의 정의와 참조가 올바르게 연결
- 주소 재배치(Relocation)
  - 프로그램이 메모리에 배치될 때 실제 주소로 수정
- 동적 링크 정보 저장
  - 실행 시 필요한 동적 라이브러리 정보를 실행 파일에 기록
- 링크 타임 최적화(LTO)
  - 불필요한 연산 제거 및 코드 실행 효율 개선

# 2.5.3.1 Static Linking vs Dynamic Linking

Heesung Oh
Copyright All rights reserved

- Static Linking
  - 빌드 시 링크
  - 라이브러리 코드가 실행 파일에 포함
  - 실행 파일 크기 증가
  - 배포 간단

- 링커에 라이브러리 추가 방법
  - Windows
    - pragma Directive 사용
      - #pragma comment(lib, "xxx.lib")
    - Visual Studio 프로젝트 설정




  - Android
    - .mk 파일: LOCAL_STATIC_LIBRARIES := libmath

# 2.5.3.1 Static Linking vs Dynamic Linking

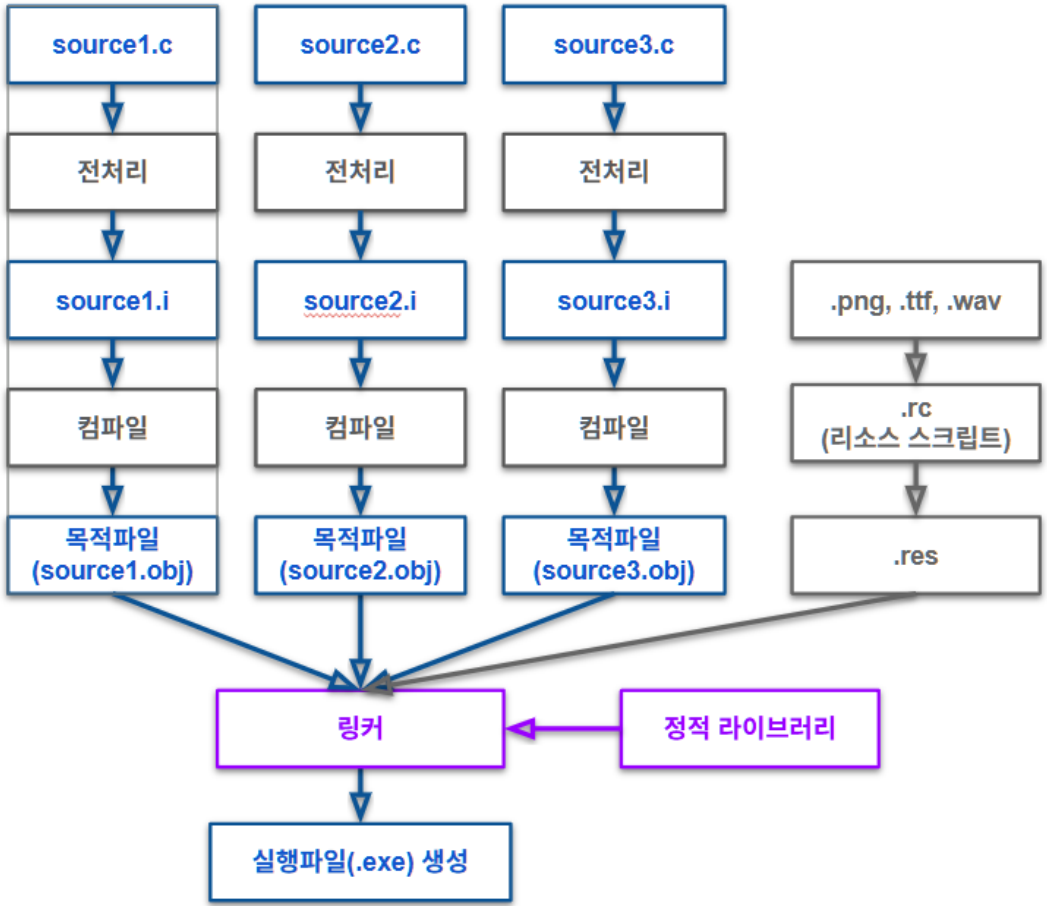
Heesung Oh
Copyright All rights reserved

- Dynamic Linking
  - 실행 시 링크
  - 실행 시 DLL/Shared Library 를 로드
  - 실행 파일 크기 작음
  - 여러 프로그램이 라이브러리 공유
- 암시적 동적 링킹 (Implicit Linking)
  - 프로그램 시작 시 OS가 DLL을 자동 로드
  - 실행 파일의 Import Table을 이용
- 명시적 동적 링킹 (Explicit Linking)
  - 프로그램이 직접 DLL/Shared Library 를 로드
  - Windows
    - LoadLibrary("xxx.dll");
  - NDK/IOS
    - dlopen("libxxx.so", RTLD_LAZY)

# 2.5.4 Windows 빌드 완성 (Final Build)

Heesung Oh
Copyright All rights reserved

- 빌드(Build)
  - 전처리 → 컴파일 → 링크 → exe
  - 실행 가능한 프로그램 생성
- 빌드에 포함되는 요소
  - 소스 코드
  - 라이브러리
  - 리소스(이미지, 폰트, 사운드 등)
- Windows 리소스 처리
  - .rc → .res → 실행 파일 포함
- 결과
  - .exe + 실행 리소스

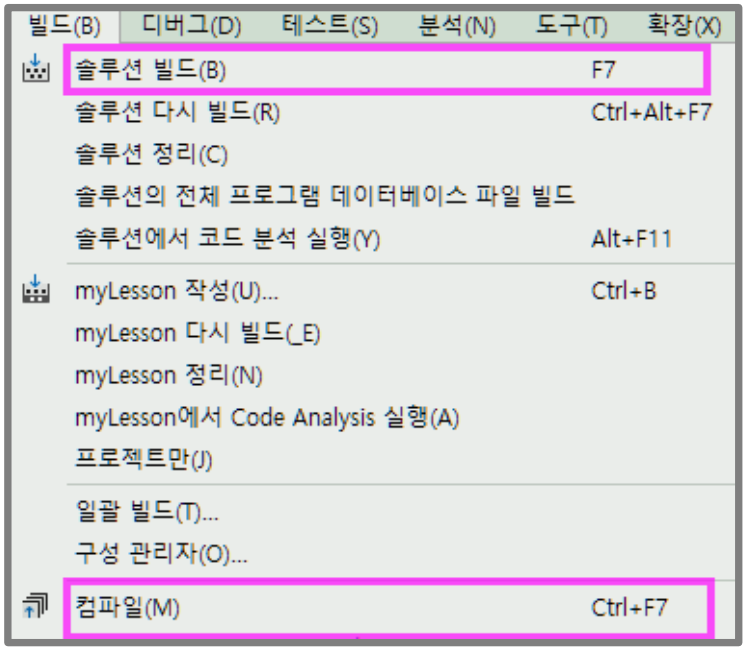


# 2.5.4.1 Visual Studio 기능

Heesung Oh
Copyright All rights reserved

- IDE (Integrated Development Environment)
  - 소스 코드 작성, 컴파일, 디버깅 등을 하나의 환경에서 수행할 수 있는 개발 도구
  - Editor + 전처리기 + 컴파일러 + 링크 + 디버깅 기능

- Visual studio 솔루션 빌드 / 컴파일



# 2.6 주석(Comment)

Heesung Oh
Copyright All rights reserved

- 주석
  - 프로그램에 대한 설명을 작성하는 문장
  - 코드 의도 설명 → 가독성 및 유지보수 향상
  - 컴파일러는 주석을 무시

- 라인 주석 (Line Comment)
  - 한 줄 주석
  - // 로 시작

- 블록 주석(Block Comment)
  - 여러 줄 주석
  - /* 로 시작, */로 종결

# 2.6 주석(Comment)

Heesung Oh
Copyright All rights reserved

- **라인 주석 예**

```
// 이력서를 출력해 봅시다.

// 자기 소개서를 작성 해봅시다
```

- **블록 주석 예**

```
/*
여러 줄에 걸쳐 설명을 작성할 수 있는 주석입니다.
프로그램의 기능이나 중요한 내용을 문서화할 때
사용됩니다.
// 라인 주석이 있어도 됩니다.
*/
```

Heesung Oh
Copyright All rights reserved

- 이력서 예제

```
#include <stdio.h>

int main(void)
{
    printf("+++++++++++++++++++++++++++++++++++++++++++++++++++++\n");
    printf("이력서\n");
    printf("+++++++++++++++++++++++++++++++++++++++++++++++++++++\n");

    printf("성  명: %s \n", "오세명");
    printf("나  이: %d \n", 20);
    printf("주  소: %s \n", "서울시 개발구 게임동");
    printf("연락처: %s-%s-%s \n", "010", "1245", "6789");

    return 0;
}
```

# 2.7.1 전처리 지시문 #include

Heesung Oh
Copyright All rights reserved

- C 언어 함수 사용 규칙
  - 함수 사용 시 함수 원형(Function Prototype) 필요
  - #include : 함수 선언이 있는 헤더 파일(.h) 포함

- #include
  - 해당 파일을 소스 코드에 포함
  - #include 〈파일명〉
    - 전처리기가 표준 라이브러리 경로에서 파일을 찾음
  - #include "파일명"
    - 전처리기가 현재 프로젝트 폴더 또는 사용자 지정 경로에서 파일을 찾음

- #include 〈stdio.h〉
  - 표준 라이브러리 경로의 stdio.h를 소스 코드에 포함
  - stdio.h : 표준 입출력(standard input output) 헤더파일

# 2.7.1.1 #include "" 〈〉 검색 순서

Heesung Oh
Copyright All rights reserved

- #include "", 〈〉 검색 순서
  - #include "file.h"
  - 현재 소스 파일 폴더에서 먼저 검색
  - 없으면 표준 라이브러리 경로에서 검색

- #include 〈file.h〉
  - 표준 라이브러리 경로에서 검색

# 2.7.2 printf() 함수

Heesung Oh
Copyright All rights reserved

- printf()
  - 문자열을 화면(콘솔)에 출력하는 C 표준 라이브러리 함수
  - 형식 지정자(format specifier)를 사용하여 변수 값 출력
  - 가장 많이 사용하는 함수 중 하나로
    프로그램 상태 확인 및 디버깅에 자주 사용

- 사용 전 헤더 파일 포함
  - #include 〈stdio.h〉

# 2.7.3 형식 지정자

Heesung Oh
Copyright All rights reserved

- 형식 지정자
  - 변수 값을 지정된 형식으로 출력
  - 형식 지정자와 인수(argument)의 개수는 반드시 일치

- 주요 형식 지정자
  - %d : 부호 있는 정수
  - %u : 부호 없는 정수
  - %c : 문자
  - %s : 문자열
  - %f : 실수(부동소수점)
  - %x : 16진수 (a~f)
  - %X : 16진수 (A~F)
  - %p : 주소
  - %zu : size_t (sizeof 결과 출력)

# 2.7.3 형식 지정자

Heesung Oh
Copyright All rights reserved

- 형식 지정자 실수 방지 요령
  - 포맷 문자열과 인수를 분리하여 개수 파악

```
printf("성   명: %s \n"
            , "오세명"     );
```

```
printf("나   이: %d \n"
            ,    20        );
```

```
printf("연락처: %s-%s-%s \n"
            , "010", "1245", "6789");
```

# 2.7.4 출력 옵션 (Format Options)

Heesung Oh
Copyright All rights reserved

- %와 형식 문자 사이 옵션 추가로 출력 형식 조정

- 폭(width)
  - 최소 출력 칸 수 지정
    - printf("%5d", 42) → "  42"

- 정밀도(precision)
  - 실수 : 소수점 이하 자릿수
  - 문자열 : 출력 길이 제한
    - printf("%.2f", 3.14159) → "3.14"
    - printf("%.5s", "HelloWorld") → "Hello"

# 2.7.4 출력 옵션 (Format Options)

Heesung Oh
Copyright All rights reserved

- 왼쪽 정렬 (-)
  - printf("%-10s", "Hi") → "Hi  "
- 0 채우기 (0)
  - printf("%05d", 42) → "00042"
- 기호 출력 (+)
  - printf("%+d", 7) → "+7"
- 진법 접두사 (#)
  - printf("%#x", 255) → "0xff"

# 2.7.5 이스케이프 시퀀스 (Escape Sequence)

Heesung Oh
Copyright All rights reserved

- 이스케이프 시퀀스
  - 제어 문자: 문자 출력 동작 제어
  - 역 슬래시(back slash: '\') 로 시작

- 주요 이스케이프 시퀀스

| 시퀀스 | 의미 | 설명 |
|---|---|---|
| \n | 줄 바꿈 (newline) | 다음 줄의 처음으로 이동 |
| \r | 캐리지 리턴 (carriage return) | 현재 줄의 맨 앞으로 이동 |
| \t | 수평 탭 (horizontal tab) | 일정 간격 탭 크기만큼 오른쪽 이동 |
| \v | 수직 탭 (vertical tab) | 수직 간격 이동 (환경에 따라 동작 다름) |
| \\ | 역 슬래시(\) 출력 | 역 슬래시(\) 문자 출력 |
| \" | 겹 따옴표(") 출력 | 문자열 안에서 큰따옴표(") 문자 출력 |
| \' | 홑 따옴표(') 출력 | 문자/문자열에서 작은따옴표(') 출력 |
| \b | 백스페이스(backspace) | 커서를 한 칸 왼쪽으로 이동 |
| \0 | 널 문자(null character) | 문자열 종료 문자, 값은 0 |

# 학습 정리

Heesung Oh
Copyright All rights reserved

- Visual Studio에서 C 프로젝트 생성
- main 함수
- 주석(Comment)
  - 라인 주석
  - 블록 주석
- printf 함수
  - #include <stdio.h>
  - 형식 지정자
    - %c, %d, %s, %f, %X %p
  - 이스케이프 시퀀스
    - \n, \t, \\, \", \', \0

# 실습 과제

Heesung Oh
Copyright All rights reserved

- 프로젝트 이름 규칙
  - 숫자, 언더스코어(_), 영문으로 만 작성. 공백과 한글은 허용 안함
  - 학번_영문이름_ProjectName
  - Ex) 9001122_JohnDoe_MyHistory

- 과제1: 이력서 작성 출력
  - 이력서용 프로젝트를 만들고, printf 함수를 사용해서 이력서 작성
  - 이름/나이/주소(시군구까지만)/연락처는 반드시 포함
  - 학력/자격증/군필/취업 또는 아르바이트 등 이력이 될만한 모든 내용을 적어보자.
  - 포맷 옵션을 사용해서 줄을 맞춰 출력해보자.

- 과제2: 자기소개서 작성 출력
  - 자기 소개서용 프로젝트를 만들고, printf 함수를 사용해서 자기 소개서 작성
  - 이름/잘하는 것/장단점 을 반드시 포함
  - 포맷 옵션을 사용해서 줄을 맞춰 출력.